



UNIVERSIDADE DO SUL DE SANTA CATARINA
AUTORIA DO TRABALHO

TÍTULO DO TRABALHO: A3-GESTÃO E QUALIDADE DE SOFTWARE

Cidade: Palhoça-SC

Ano: 2026

AUTORIA DO TRABALHO

**TÍTULO DO TRABALHO: A3-GESTÃO E
QUALIDADE DE SOFTWARE**

Trabalho de Conclusão de Semestre
apresentado ao Curso de análise e
desenvolvimento de sistemas da
Universidade do Sul de Santa Catarina

Orientador: Prof. Jorge e Silvia

Cidade: Palhoça-SC
Ano: 2026

Universidade do Sul de Santa Catarina- Unisul Unidade

Curricular: **Gestão e qualidade de software**

Alunos: Antonio Victor Iaroseski Segala - RA:1072417746

Isadora Luchtenberg Fernandes - RA:1072417018

Lucas Rengel - RA: 10724110009

SUMÁRIO

Introdução.....	01
Desenvolvimento.....	02
Objetivo do Plano de Testes.....	02
Escopo do Teste.....	03
Contexto do Sistema Legado.....	04
Estratégia de Testes.....	05
Ambiente de Teste.....	06
Dados de Teste.....	07
Ferramentas de Teste.....	08
Cronograma.....	09
Responsabilidades.....	10
Critérios de Aceitação	12
Gestão de Defeitos.....	14
Relatórios e Métricas.....	15
Controle de Versão	17
Links dos projetos no GitHub	19
conclusão	20
Referências	21

1- INTRODUÇÃO

A qualidade de software é um dos principais fatores para garantir que uma aplicação atenda às necessidades dos usuários de forma eficiente, confiável e segura. Nesse contexto, a realização de testes sistemáticos torna-se fundamental para identificar falhas, validar requisitos e assegurar o correto funcionamento das funcionalidades desenvolvidas.

Este Plano de Testes foi elaborado para o Sistema de Controle de Estoque, uma aplicação desenvolvida para auxiliar no gerenciamento de produtos, categorias, movimentações de entrada e saída de estoque, além da emissão de relatórios gerenciais. O documento apresenta as diretrizes, estratégias, recursos e critérios que serão utilizados durante o processo de verificação e validação do sistema.

Além de contribuir para a identificação e correção de defeitos, o plano estabelece práticas de qualidade de software, incluindo testes automatizados, integração contínua, análise estática de código e controle de versão. Essas ações têm como objetivo aumentar a confiabilidade, a manutenção e a estabilidade da aplicação, reduzindo riscos durante futuras evoluções do sistema.

Dessa forma, este documento serve como referência para o planejamento, execução e acompanhamento das atividades de teste, garantindo que o software atenda aos requisitos definidos e esteja preparado para oferecer uma experiência consistente e satisfatória aos seus usuários.

2- DESENVOLVIMENTO

2.1 Objetivo do Plano de Testes

O presente plano de testes tem como objetivo definir a estratégia, os procedimentos e os critérios utilizados para verificar a qualidade do Sistema de Controle de Estoque, garantindo que suas funcionalidades operem de forma correta, confiável e segura.

O sistema foi desenvolvido para auxiliar no gerenciamento de estoque de produtos, permitindo o cadastro de categorias, produtos, registro de movimentações de entrada e saída e a geração de relatórios gerenciais. Considerando que o software será mantido e evoluído ao longo do tempo, torna-se necessário estabelecer um processo estruturado de testes que assegure a integridade das funcionalidades existentes e reduza a ocorrência de falhas.

A execução dos testes busca validar os requisitos funcionais do sistema, identificar defeitos, melhorar a confiabilidade da aplicação e garantir que futuras alterações não comprometam funcionalidades já implementadas. Além disso, serão utilizadas práticas de integração contínua, automação de testes e análise estática de código para elevar a qualidade do software e atender aos critérios definidos para o projeto.

Por meio deste plano, pretende-se obter maior cobertura do código-fonte, aumentar a manutenção do sistema e assegurar que o software esteja apto para atender às necessidades dos usuários de forma consistente e eficiente.

2.2 Escopo do Teste

Os testes serão realizados nas funcionalidades principais do Sistema de Controle de Estoque, contemplando os módulos responsáveis pelo gerenciamento de categorias, produtos, movimentações de estoque e geração de relatórios gerenciais.

Serão testadas as seguintes funcionalidades:

- Cadastro de categorias;
- Edição de categorias;
- Remoção de categorias;
- Cadastro de produtos;
- Edição de produtos;
- Remoção de produtos;
- Ajuste de preços por percentual;
- Registro de movimentações de entrada de estoque;
- Registro de movimentações de saída de estoque;
- Edição de movimentações;
- Remoção de movimentações;
- Geração de relatório de lista de preços;
- Geração de relatório de balanço de estoque;
- Geração de relatório de produtos abaixo do estoque mínimo;
- Geração de relatório de produtos agrupados por categoria;
- Geração de relatório de maiores movimentações.

Funcionalidades Não Contempladas pelos Testes

Não serão realizados testes de desempenho, carga, estresse e segurança avançada, pois o foco desta atividade está concentrado na aplicação de práticas de qualidade de software voltadas à automação de testes, integração contínua e análise estática de código. Também não serão realizados testes formais de usabilidade e acessibilidade, uma vez que o objetivo principal do trabalho é validar as regras de negócio e a qualidade do código-fonte do sistema legado.

2.3 Contexto do Sistema Legado

O Sistema de Controle de Estoque foi desenvolvido originalmente como parte da atividade A3 da Unidade Curricular de Sistemas Distribuídos e Mobile. O sistema tem como finalidade auxiliar no gerenciamento de estoques por meio do cadastro de categorias e produtos, controle de movimentações de entrada e saída e geração de relatórios gerenciais para apoio à tomada de decisão.

A arquitetura da aplicação é composta por um frontend desenvolvido utilizando React, TypeScript e Vite, responsável pela interface com o usuário, e um backend desenvolvido em Java 17 utilizando o framework Spring Boot, responsável pelo processamento das regras de negócio e acesso aos dados. O armazenamento das informações é realizado em banco de dados MySQL.

Entre as principais funcionalidades disponibilizadas pelo sistema estão o cadastro, edição e remoção de categorias e produtos, o ajuste percentual de preços, o registro e gerenciamento de movimentações de estoque e a emissão de relatórios gerenciais, incluindo lista de preços, balanço de estoque, produtos abaixo do estoque mínimo, produtos por categoria e maiores movimentações.

Por se tratar de um sistema legado utilizado nesta atividade, o software foi desenvolvido anteriormente e será submetido a um processo de melhoria de qualidade. O foco desta etapa será a implementação de testes automatizados, integração contínua, análise estática de código e gestão de configuração, buscando aumentar a confiabilidade, a manutenibilidade e a qualidade geral da aplicação.

Até o início deste trabalho não foram realizadas manutenções significativas ou evoluções funcionais após a entrega original do projeto. Dessa forma, o sistema encontra-se em sua versão inicial, servindo como base para a aplicação das práticas de qualidade de software propostas nesta atividade.

Documentação Existente

O sistema possui documentação básica disponível por meio do arquivo README presente nos repositórios do projeto. Essa documentação contém instruções de instalação, configuração do ambiente e execução da aplicação. Como se trata de um projeto acadêmico previamente desenvolvido, a compreensão detalhada das funcionalidades foi complementada por meio da análise do código-fonte e da estrutura do banco de dados.

2.4 Estratégia de Testes

A estratégia de testes adotada para o Sistema de Controle de Estoque tem como objetivo verificar o correto funcionamento das funcionalidades implementadas e garantir a qualidade do software durante sua evolução.

Serão utilizados principalmente testes unitários para validar as regras de negócio implementadas no backend da aplicação. Os testes serão desenvolvidos utilizando frameworks apropriados para o ecossistema Java e executados automaticamente por meio de integração contínua.

Também serão realizados testes de integração para validar a comunicação entre as camadas da aplicação e o acesso ao banco de dados, garantindo que os dados sejam processados e persistidos corretamente.

Os testes contemplarão os módulos de categorias, produtos, movimentações e relatórios, verificando cenários de sucesso, validações de regras de negócio e tratamento de possíveis falhas.

Além dos testes automatizados, será realizada análise estática do código-fonte utilizando ferramentas de inspeção de qualidade, permitindo identificar problemas relacionados à manutenibilidade, confiabilidade, segurança, duplicação de código e complexidade.

Como o sistema possui documentação limitada por se tratar de um projeto acadêmico previamente desenvolvido, a compreensão das funcionalidades será realizada por meio da análise do código-fonte, da estrutura do banco de dados e da execução prática da aplicação. Essa abordagem permitirá identificar os comportamentos esperados do sistema e elaborar casos de teste adequados para cada funcionalidade.

Os testes serão executados continuamente durante o desenvolvimento, utilizando integração contínua para garantir que novas alterações não comprometam funcionalidades já existentes e que os critérios de qualidade definidos para o projeto sejam atendidos.

Também serão realizados testes de regressão por meio da execução automática da suíte de testes sempre que novas alterações forem enviadas ao repositório. Não serão realizados testes de desempenho e segurança aprofundados, uma vez que tais atividades não fazem parte do escopo principal desta etapa do projeto. Entretanto, aspectos relacionados à confiabilidade, manutenibilidade e segurança básica do código serão avaliados por meio da análise estática realizada com o SonarCloud.

2.5 Ambiente de Teste

Os testes do Sistema de Controle de Estoque serão executados em ambiente de desenvolvimento controlado, reproduzindo as condições necessárias para validação das funcionalidades da aplicação.

O ambiente utilizado possui as seguintes características:

Hardware e Sistema Operacional

- Sistema Operacional: Windows 11;
- Computadores pessoais dos integrantes da equipe.

Backend

- Linguagem: Java 17;
- Framework: Spring Boot;
- IDE: IntelliJ IDEA;
- Porta de execução: 8080.

Frontend

- Framework/Biblioteca: React;
- Linguagem: TypeScript;
- Ferramenta de build: Vite;
- IDE: Visual Studio Code;
- Porta de execução: 5173.

Banco de Dados

- Sistema Gerenciador de Banco de Dados: MySQL;
- Versão: 8.0.

Para execução dos testes automatizados será utilizado o ambiente configurado no projeto, garantindo que os resultados obtidos sejam consistentes com o comportamento esperado da aplicação. Também será utilizada uma pipeline de integração contínua para executar os testes automaticamente a cada atualização relevante do código-fonte.

Como o sistema foi desenvolvido recentemente e não possui ambiente legado complexo ou dependências externas críticas, não será necessária uma estratégia específica de replicação de infraestrutura. O ambiente de testes será mantido alinhado ao ambiente de desenvolvimento utilizado pela equipe.

2.6 Dados de Teste

Os dados de teste serão gerados pela própria equipe com o objetivo de simular situações reais de utilização do Sistema de Controle de Estoque. Essa abordagem permite validar as funcionalidades sem a necessidade de utilizar dados reais ou sensíveis.

Serão utilizadas categorias representativas de diferentes segmentos de mercado, tais como:

- Eletrônicos;
- Alimentos;
- Vestuário.

Para os produtos serão cadastrados exemplos como:

- Notebook;
- Mouse;
- Arroz;
- Camiseta.

As movimentações de estoque incluirão operações de entrada e saída de produtos em diferentes quantidades, permitindo validar os cálculos de saldo, atualização de estoque e geração de relatórios gerenciais.

Também serão criados cenários específicos para testar situações de exceção e regras de negócio, incluindo:

- Produtos com estoque abaixo do mínimo;
- Produtos sem movimentação recente;
- Ajustes de preços utilizando percentuais positivos e negativos;
- Tentativas de cadastro com dados inválidos ou incompletos.

Os dados utilizados nos testes serão armazenados em ambiente de desenvolvimento e poderão ser recriados sempre que necessário, garantindo a repetibilidade dos testes e a consistência dos resultados obtidos.

Como o sistema não utiliza dados pessoais ou informações confidenciais, não será necessário aplicar procedimentos específicos de anonimização ou mascaramento de dados durante a execução dos testes.

2.7 Ferramentas de Teste

Para garantir a qualidade do Sistema de Controle de Estoque, serão utilizadas ferramentas que auxiliam na automação de testes, medição de cobertura, integração contínua, análise estática de código e gestão de configuração.

As principais ferramentas utilizadas no projeto são:

- **GitHub:** utilizado para controle de versão e gerenciamento do código-fonte.
- **Maven:** utilizado para gerenciamento de dependências, build e execução dos testes automatizados.
- **JUnit:** utilizado para desenvolvimento e execução dos testes unitários do backend.
- **GitHub Actions:** utilizado para automatizar a execução dos testes sempre que houver alterações no repositório.
- **SonarCloud:** utilizado para análise estática do código-fonte, identificação de vulnerabilidades, problemas de manutenção, duplicações e acompanhamento dos indicadores de qualidade do projeto.
- **JaCoCo:** utilizado para geração e análise da cobertura de testes do backend, permitindo medir o percentual do código exercitado pelos testes automatizados.

A escolha dessas ferramentas foi realizada devido à sua ampla utilização no mercado de desenvolvimento de software, facilidade de integração com projetos Java Spring Boot e compatibilidade com práticas modernas de Integração Contínua (CI) e Entrega Contínua (CD).

A automação dos testes permitirá a validação contínua das funcionalidades implementadas, reduzindo a probabilidade de regressões e aumentando a confiabilidade do sistema. Já a análise estática do código auxiliará na identificação antecipada de problemas de qualidade, contribuindo para a evolução e manutenção do software.

2.8 Cronograma

Atividade	Período
Análise do sistema legado	Semana 1
Elaboração do plano de testes	Semana 1
Configuração do repositório e organização das tarefas	Semana 1
Desenvolvimento dos testes unitários	Semana 2
Execução e correção dos testes	Semana 2
Configuração da integração contínua (GitHub Actions)	Semana 3
Configuração da análise estática (SonarCloud)	Semana 3
Correção dos problemas apontados pelo Quality Gate	Semana 3
Elaboração do relatório final	Semana 4
Preparação da apresentação	Semana 4

- Conclusão da análise do sistema legado.
- Finalização dos testes unitários.
- Cobertura mínima de 75% atingida.
- Configuração do GitHub Actions.
- Aprovação do Quality Gate no SonarCloud.
- Entrega do relatório final.

2.9 Responsabilidades

Para garantir a organização e o desenvolvimento adequado das atividades previstas neste projeto, foram definidos papéis e responsabilidades para cada integrante da equipe. Embora cada membro possua uma função principal, todos os integrantes participarão do desenvolvimento de código e da implementação de testes unitários.

Integrante	Papel Principal	Responsabilidades
Lucas	Líder da Equipe e Especialista DevOps	Coordenação das atividades, acompanhamento do cronograma, organização do repositório, configuração da integração contínua e apoio no desenvolvimento dos testes.
Isa	Desenvolvedora	Implementação e manutenção das funcionalidades do sistema, correção de defeitos identificados e desenvolvimento de testes unitários.
Antonio	Testador	Planejamento dos testes, validação dos resultados, registro de defeitos e desenvolvimento de testes unitários.

A distribuição das funcionalidades para implementação dos testes unitários será realizada de forma equilibrada entre os integrantes, garantindo participação ativa de todos os membros do grupo e contribuindo para a obtenção da cobertura mínima exigida para o projeto.

Aluno	Funcionalidade	Link
Lucas Rengel	Movimentações	https://github.com/lucasrengel/A3Back/blob/master/src/test/java/unisul/a3/service/MovimentacaoServiceTest.java
Lucas Rengel	Relatórios	https://github.com/lucasrengel/A3Back/blob/master/src/test/java/unisul/a3/service/RelatorioServiceTest.java
Isadora Fernandes	Produtos	https://github.com/lucasrengel/A3Back/blob/master/src/test/java/unisul/a3/service/ProdutoServiceTest.java

Isadora Fernandes	Produto Controller	https://github.com/lucasrengel/A3Back/blob/master/src/test/java/unisul/a3/controller/ProdutoControllerTest.java
Antonio Segala	Categorias	https://github.com/lucasrengel/A3Back/blob/master/src/test/java/unisul/a3/service/CategoriaServiceTest.java
Antonio Segala	Categoria Controller	https://github.com/lucasrengel/A3Back/blob/master/src/test/java/unisul/a3/controller/CategoriaControllerTest.java

2.10 Critérios de Aceitação

Os testes serão considerados satisfatórios quando os critérios descritos a seguir forem atendidos.

Cobertura de Código

- Obter cobertura mínima de 75%.
- Resultado obtido: 80% de cobertura de código utilizando a ferramenta JaCoCo.
- Garantir que as principais regras de negócio estejam contempladas pelos testes unitários.

Execução dos Testes

- Todos os testes automatizados devem ser executados com sucesso.
- Não devem existir falhas críticas ou bloqueadoras nos testes implementados.
- Os testes devem validar os comportamentos esperados das funcionalidades do sistema.

Funcionalidades do Sistema

As seguintes funcionalidades deverão apresentar comportamento correto durante a execução dos testes:

Categorias

- Cadastro de categorias;
- Edição de categorias;
- Remoção de categorias.

Produtos

- Cadastro de produtos;
- Edição de produtos;
- Remoção de produtos;
- Ajuste de preços por percentual.

Movimentações

- Registro de entradas de estoque;
- Registro de saídas de estoque;
- Edição de movimentações;
- Remoção de movimentações.

Relatórios

- Relatório de lista de preços;

- Relatório de balanço de estoque;
- Relatório de produtos abaixo do estoque mínimo;
- Relatório de produtos por categoria;
- Relatório de maiores movimentações.

Integração Contínua

- Os testes automatizados devem ser executados automaticamente por meio da pipeline de integração contínua configurada no GitHub Actions.
- Alterações enviadas ao repositório não devem comprometer a execução da pipeline.

Qualidade do Código

- O projeto deve passar no Quality Gate definido pela ferramenta SonarCloud.
- Todos os issues identificados como críticos deverão ser corrigidos.
- Não deverão existir vulnerabilidades críticas de segurança.
- Os problemas de manutenibilidade e confiabilidade identificados deverão ser tratados sempre que possível.

Controle de Versão

- Todas as alterações deverão ser registradas no GitHub.
- Os commits deverão possuir mensagens claras e rastreáveis.
- Todos os integrantes deverão contribuir com código-fonte e testes unitários no repositório.

O projeto será considerado aprovado quando todos os critérios descritos anteriormente forem atendidos e a aplicação demonstrar funcionamento correto durante os testes e apresentação final.

2.11 Gestão de Defeitos

Durante o desenvolvimento e execução dos testes, os defeitos identificados serão registrados, analisados, corrigidos e validados pela equipe, garantindo a rastreabilidade das correções realizadas no projeto.

O processo de gestão de defeitos seguirá as seguintes etapas:

1. Identificação do defeito durante a execução dos testes ou análise do sistema;
2. Registro do problema utilizando o sistema de Issues do GitHub;
3. Classificação do defeito de acordo com sua gravidade e impacto no funcionamento da aplicação;
4. Correção do problema pelo integrante responsável;
5. Execução de novos testes para validação da correção;
6. Encerramento da Issue após confirmação da solução.

As correções realizadas serão registradas por meio de commits no repositório GitHub, permitindo rastrear quando o problema foi corrigido e por qual integrante da equipe.

Além dos defeitos identificados durante os testes, também serão considerados os problemas apontados pelas ferramentas de análise estática de código, especialmente aqueles relacionados à confiabilidade, manutenibilidade, segurança, duplicação e complexidade do código-fonte.

A utilização do GitHub Issues permitirá centralizar o acompanhamento dos problemas encontrados, facilitando a comunicação entre os integrantes e o controle das correções realizadas ao longo do projeto.

2.12 Relatórios e Métricas

O acompanhamento da qualidade do Sistema de Controle de Estoque foi realizado por meio de métricas coletadas durante a execução dos testes automatizados e da análise estática do código-fonte.

Os testes unitários foram desenvolvidos para as principais funcionalidades do sistema, abrangendo os módulos de categorias, produtos, movimentações e relatórios. Para cada funcionalidade foi criado um conjunto de testes automatizados responsável por validar os comportamentos esperados e reduzir a ocorrência de falhas durante futuras manutenções.

Funcionalidades Cobertas pelos Testes

Funcionalidade	Arquivo de Teste
Categorias	CategoriaServiceTest.java
Categorias (Controller)	CategoriaControllerTest.java
Produtos	ProdutoServiceTest.java
Produtos (Controller)	ProdutoControllerTest.java
Movimentações	MovimentacaoServiceTest.java
Movimentações (Controller)	MovimentacaoControllerTest.java
Relatórios	RelatorioServiceTest.java
Relatórios (Controller)	RelatorioControllerTest.java
Conexão com Banco de Dados	DatabasConnectionTest.java
Inicialização do Banco de Dados	DatabasInitializerTest.java

A cobertura dos testes foi monitorada utilizando a ferramenta JaCoCo, permitindo verificar o percentual do código-fonte exercido pelos testes automatizados. Como requisito da atividade, foi estabelecida uma cobertura mínima de 75% do backend da aplicação.

Análise Estática do Código-Fonte

A qualidade do código-fonte foi avaliada por meio da plataforma SonarCloud, que realizou inspeções relacionadas às seguintes métricas:

- Segurança (Security);
- Confiabilidade (Reliability);
- Manutenibilidade (Maintainability);
- Duplicação de Código (Duplications);
- Complexidade (Complexity);
- Cobertura de Testes (Coverage).

Durante a análise inicial foram identificados problemas relacionados à qualidade do código, incluindo oportunidades de melhoria na manutenibilidade, duplicações de código, cobertura insuficiente em determinados trechos da aplicação e ajustes recomendados pela ferramenta de análise estática.

Para atender aos critérios estabelecidos pelo Quality Gate do SonarCloud foram realizadas as seguintes ações:

- Criação de novos testes unitários para aumentar a cobertura do código;
- Refatoração de trechos identificados como duplicados;
- Ajustes em métodos com baixa manutenibilidade;
- Correção de problemas apontados pela análise estática;
- Revisão de práticas de codificação para aumentar a confiabilidade da aplicação.

Após a implementação das correções, o projeto passou a atender aos requisitos definidos pelo SonarCloud, obtendo aprovação no Quality Gate e eliminando problemas críticos identificados durante as análises.

Resultados Obtidos

- Cobertura de testes: **80%**;
- Pipeline GitHub Actions configurada para execução automática dos testes a cada push realizado no repositório;
- Integração com SonarCloud realizada com sucesso;
- Quality Gate aprovado;
- Ausência de vulnerabilidades críticas;
- Ausência de issues críticas de confiabilidade e manutenibilidade;
- Análise estática executada com sucesso e dentro dos critérios definidos para o projeto.

Essas métricas demonstram a evolução da qualidade do software ao longo do desenvolvimento, contribuindo para um sistema mais confiável, manutenível e preparado para futuras evoluções.

2.13 Controle de Versão

O controle de versão do Sistema de Controle de Estoque será realizado por meio da plataforma GitHub, permitindo o gerenciamento das alterações realizadas no código-fonte, rastreabilidade das modificações e colaboração entre os integrantes da equipe.

O projeto utiliza a branch principal **master**, conforme definido para o desenvolvimento e manutenção do software. Todas as alterações implementadas serão registradas por meio de commits frequentes e organizados, permitindo acompanhar a evolução do sistema ao longo do projeto.

A estrutura do repositório contempla os principais elementos necessários para a organização do projeto, incluindo:

- Código-fonte da aplicação;
- Arquivos de configuração;
- Documentação do projeto;
- Arquivos relacionados aos testes automatizados;
- README com instruções de utilização.

Para manter um histórico de alterações claro e compreensível, será utilizada uma convenção padronizada para as mensagens de commit. Alguns exemplos incluem:

- **feat:** adiciona cadastro de produto
- **fix:** corrige cálculo de movimentação
- **test:** adiciona testes da categoria
- **docs:** atualiza documentação
- **refactor:** melhora organização do código

Essa padronização facilita a identificação das alterações realizadas e contribui para a rastreabilidade das modificações efetuadas por cada integrante da equipe.

O repositório também servirá como evidência da participação dos integrantes no desenvolvimento do projeto, permitindo verificar as contribuições individuais por meio do histórico de commit

```
├── README.md
├── pom.xml
├── src
│   ├── main
│   └── test
└── .github/workflows
```

2.14 Links dos projetos no GitHub

Repositórios:

Backend:

<https://github.com/lucasrengel/A3Back>

Frontend:

<https://github.com/lucasrengel/A3Front>

SonarCloud:

https://sonarcloud.io/project/overview?id=lucasrengel_A3Back

2.15- Conclusão

A elaboração deste Plano de Testes permitiu definir e aplicar de forma estruturada os processos, critérios e ferramentas utilizados para avaliar a qualidade do Sistema de Controle de Estoque. As estratégias estabelecidas contribuíram para validar o correto funcionamento das funcionalidades implementadas, garantindo maior confiabilidade e aderência aos requisitos definidos para o projeto.

Durante o desenvolvimento, foram utilizados testes automatizados com JUnit, análise de cobertura com JaCoCo, integração contínua por meio do GitHub Actions e análise estática de código utilizando SonarCloud. Essas práticas possibilitaram a identificação e correção antecipada de falhas, a redução de riscos de regressão e a melhoria contínua da qualidade do software.

Como resultado, o projeto alcançou 80% de cobertura de código, obteve aprovação no Quality Gate do SonarCloud e passou a executar automaticamente os testes a cada atualização do repositório. Dessa forma, os objetivos propostos foram atingidos, resultando em um sistema mais robusto, confiável e de fácil manutenção, além de proporcionar aos integrantes experiência prática na utilização de técnicas e ferramentas amplamente empregadas no mercado de desenvolvimento de software

3.0- REFERÊNCIAS

GITHUB. GitHub Actions Documentation. Disponível em:

<https://docs.github.com/actions>

SONARSOURCE. SonarCloud Documentation. Disponível em:

<https://docs.sonarcloud.io>

JUNIT TEAM. JUnit 5 User Guide. Disponível em:

<https://junit.org/junit5/docs/current/user-guide>

JACOCO. Java Code Coverage Library. Disponível em:

<https://www.jacoco.org/jacoco>